

Splitting generated.mli

We want one `.mli` per module instead of one file holding all of them. Referential integrity is what makes that hard, because the foreign key and the join form a cycle — and OCaml compilation units cannot be mutually recursive. Two ways out, both verified.

Checked with `ocamlc 5.5.0` Files `examples/split-mli/` Status nothing implemented yet
Source claude.ai/code/artifact/0380906b-a557-4fab-a501-444c2d58a4ce

01

Why the file can't just be cut apart

Today every module lands in one file, so `module rec` lets them refer to each other freely. As soon as the document contains a collection, those references form a genuine cycle:

```
(* generated.mli, as emitted today *)
module rec Xs : sig
  type t = { id : id; parent_id : Root.id; idx : int; n : int }   (* Xs → Root *)
end
and Root : sig
  val xs : t -> Xs.t list                                       (* Root → Xs *)
end
```

`Xs` names `Root.id` — that is the foreign key. `Root` names `Xs.t` — that is the join. **The cycle is the referential integrity.** It is fine inside one file and impossible across two, so splitting means deliberately removing one of the two arrows.

02

Cutting it apart unchanged fails both ways round

There is no ordering that rescues it. Each attempt fails on the half of the cycle the other one satisfied:

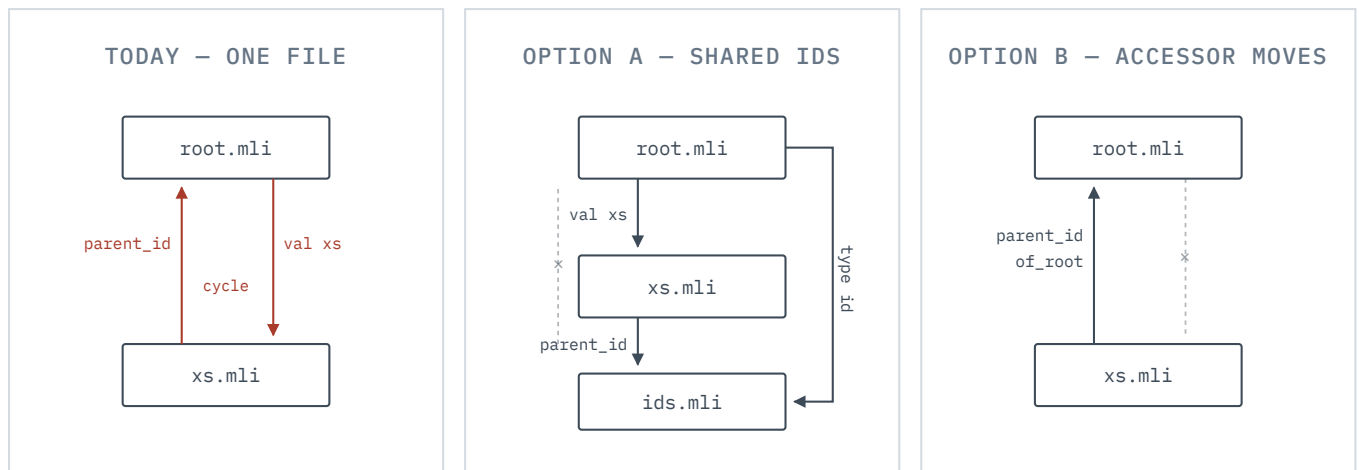
```
ocamlc -c b.mli xs.mli root.mli
```

```
File "xs.mli", line 2:
type t = { id : id; parent_id : Root.id; ... }
                                ^^^^^
Error: Unbound module Root
```

```
ocamlc -c b.mli root.mli xs.mli
```

```
File "root.mli", line 5:
val xs : t -> Xs.t list
                ^^
Error: Unbound module Xs
```

The edge each option removes



`root.mli` and `xs.mli` sit in the same place in all three panels, so the difference is the arrow that disappears. A dashed line with `x` marks the reference each option removes. Red marks the pair of references that form the cycle; every solid arrow that remains points one way only.

The design note's own example can't decide this

`{ "a": 1, "b": { "c": 1, "d": 2 } }` — §2.1 Step 1 — has a nested object but **no collection**, so there is no cycle in it.

Split naively, under option A, under option B: **all three compile**. The note's example is in the repo as the reference everything is measured against, but the comparison below uses the same document with one repeated substructure added, because that is the only shape where the two proposals differ.

The document used throughout the rest of this page:

```
{ "a": 1, "b": { "c": 1, "d": 2 }, "xs": [ { "n": 1 }, { "n": 2 } ] }
```

What each one actually emits

The `b.mli` module is identical in all three and is left out here. Full files are in `examples/split-mli/d2-with-collection/`.

Today

ONE FILE

Option A

COMPILES

Option B

COMPILES

generated.mli

```
module rec Xs : sig
  type id
  type t = { id : id;
    parent_id : Root.id;
    idx : int; n : int }
  val get : id -> t
end

and B : sig ... end

and Root : sig
  type id
  type t = { id : id; a : int;
    b : B.id }
  val get : id -> t
  val b : t -> B.t
  val xs : t -> Xs.t list
end
```

order – n/a, single unit

ids.mli

```
(* every table's key type *)
type xs
type b
type root
```

xs.mli

```
type id = Ids.xs
type t = { id : id;
  parent_id : Ids.root;
  idx : int; n : int }
val get : id -> t
```

root.mli

```
type id = Ids.root
type t = { id : id; a : int;
  b : B.id }
val get : id -> t
val b : t -> B.t
val xs : t -> Xs.t list
```

order – ids, xs, b, root
deepest-first, unchanged

root.mli

```
type id
type t = { id : id; a : int;
  b : B.id }
val get : id -> t
val b : t -> B.t
```

xs.mli

```
type id
type t = { id : id;
  parent_id : Root.id;
  idx : int; n : int }
val get : id -> t
val of_root : Root.id -> t list
```

order – b, root, xs
needs a topological sort

What the compiler says

Every row is `ocamlc -c` on the real files. D1 is the design note’s document; D2 adds the collection.

VARIANT	D1 – NO COLLECTION	D2 – WITH COLLECTION
naive split	COMPILES	FAILS, BOTH ORDERS
option A	COMPILES	COMPILES
option B	COMPILES	COMPILES

Referential integrity holds either way

This is the thing worth checking, and it does **not** separate the two. Both consumers below were type-checked against the generated `.cmi` files under each option.

The join, both directions

ACCEPTED ×2

use.ml — option A

```
let children (r : Root.t) : Xs.t list =  
  Root.xs r  
let owner (x : Xs.t) : Root.t =  
  Root.get x.Xs.parent_id
```

use.ml — option B

```
let children (r : Root.t) : Xs.t list =  
  Xs.of_root r.Root.id  
let owner (x : Xs.t) : Root.t =  
  Root.get x.Xs.parent_id
```

Forging a key

REJECTED ×2

forge.ml — option A

```
let bad : Ids.root = 0  
Error: The constant 0 has type int  
      but an expression was expected of  
      type Ids.root
```

forge.ml — option B

```
let bad : Root.id = 0  
Error: The constant 0 has type int  
      but an expression was expected of  
      type Root.id
```

08

What actually separates them

Option A

a shared `ids.mli`

- + Accessors stay on the parent, matching the design note's §2.1 correspondence — *joining on a foreign key becomes an accessor function*.
- + Emit order stays deepest-first, which is the order tables already come out in. No new sorting code.
- + A foreign key into a nested object still reads `b : B.id`, exactly as the note writes it. Only collection back-references change.
- + `Root.id` stays usable everywhere — it is a transparent alias for the abstract `Ids.root`.
- One extra generated file, and `Ids` becomes a reserved module name that a table called `ids` would collide with.

Option B

the accessor moves to the child

- + No extra file. Every `id` stays abstract inside its own module, exactly as now.
- + `Xs.of_root` reads like the query it lowers to — `select * from xs where parent_id = ?` — which may matter more for the Phase 2 work than it does here.
- `Root.xs r` becomes `Xs.of_root r.Root.id`. The join reads from the child, which is the opposite of where the note puts it.
- Emit order stops being deepest-first: nested objects order parent-after-child, collections order parent-before-child. Needs a real topological sort instead of the current length sort.

09

What we need to decide

1. Where should the join accessor live — on the parent (`Root.xs`) or on the child (`Xs.of_root`)?
2. Is one extra generated module, `ids.mli`, acceptable in the output?
3. For the query-optimisation work: does `Xs.of_root : Root.id -> t list` carry more useful information than `Root.xs : t -> Xs.t list`, or less?

Everything here is hand-written to compare the two before either is built — nothing in `Tatami/` has changed. Run `examples/split-mli/check.sh` to reproduce every result on this page.